

Examen Final de GRAU-IA

(14 de enero de 2025)

Duración: 2 horas 30 minutos

1. (5 puntos) La agencia de viajes *Viajando Voy* quiere renovar su página de web y su app con un sistema inteligente de recomendación que ayude a sus usuarios registrados a escoger entre su extenso catálogo de viajes. Para hacer esta recomendación el sistema usa no solo la petición de viaje que realiza un usuario (que se identifica con un nombre de usuario y password) sino también el historial de viajes anteriores que ha hecho el usuario en esa agencia.

El catálogo de la agencia consta de destinos, y cada uno tiene asociado el nombre de la población, el país en el que se encuentra, el nivel de riesgo (del 1 a 10, tiene en cuenta la posibilidad de robos, de accidentes, de atentados), la moneda de pago, los medios de transporte disponibles (avión, tren, barco, autobús, taxi, vehículo de alquiler, etc.) los lugares de transporte (aeropuertos, puertos, estaciones de tren, estaciones de autobús, agencias de alquiler de coches, etc.) los alojamientos disponibles (por ahora solo hoteles, hostales y albergues) y las actividades (de ocio, cultura, deporte o experiencias) que se pueden hacer en ese destino.

De los lugares de transporte se guarda su nombre, dirección completa, y las coordenadas GPS, y en el caso de los aeropuertos se guarda también el código internacional de tres letras (por ej "BCN" para Barcelona). De los aviones se guarda la compañía aérea, el identificador de vuelo, el aeropuerto de origen y el aeropuerto final, las escalas (aeropuertos en los que se hace una parada intermedia en el viaje), la hora de salida y la hora de llegada, y el precio del trayecto (en euros); en el caso de los trenes se guarda la compañía, la estación de tren de origen y la estación de final, la hora de salida y la hora de llegada, y el precio del trayecto (en euros); en el caso de los autobuses se guarda la compañía, la estación de autobuses de origen y la estación de final, la hora de salida y la hora de llegada, y el precio del trayecto (en euros); en el caso de los barcos se guarda la compañía, el puerto de origen y el puerto final, las escalas (puertos en los que se hace una parada intermedia en el viaje), la hora de salida y la hora de llegada, y el precio del trayecto (en euros); en el caso de los taxis se guarda la dirección de origen, la dirección de llegada y el precio por kilómetro (en euros); en el caso de los vehículos de alquiler se guarda la oficina de alquiler de recogida, la oficina de alquiler de entrega, la fecha de recogida, la fecha de entrega, la marca y modelo del vehículo, el número de plazas, el precio base de alquiler por día, el precio por kilómetros extra recorridos y el precio del seguro de accidentes.

Cada alojamiento (hotel, hostel o albergue) tiene asociado un nombre, el número de estrellas que tiene (de 1 a 5 estrellas) y cada una de las habitaciones. Para cada habitación se guarda el identificador de habitación, el número de camas individuales, el número de camas dobles, el número máximo de personas, si tiene baño propio o no, si tiene televisión o no, si tiene wifi o no y si tiene mini-bar o no. Hay un tipo especial de alojamiento: los cruceros, que disponen de las características de Hotel y Barco a la vez, ya que son un alojamiento y también un medio de transporte.

De las actividades se tiene el nombre, la dirección, una descripción corta, el nivel de riesgo (de 1 a 10), la edad mínima y máxima para poder hacerla, y si es una actividad para hacer de forma individual, en pareja o en grupos. Hay cuatro tipos de actividades: de ocio, culturales, deporte y experiencias. Como actividades de ocio se tiene la playa y los parques temáticos (de estos se guarda hora de apertura, hora de cierre y precio); de actividades culturales se tienen los museos y los monumentos (y de ambos se guarda hora de apertura, hora de cierre y precio); de actividades de deporte se tiene el trekking y el esquí (y en ambas se tiene asociado el nivel técnico necesario, de 1 a 10); y se tiene tres tipos de experiencias: románticas, culinarias o safari (en todas ellas se guarda la hora de inicio, la duración y el precio).

Una petición de viaje introducida por el usuario incluye un precio máximo, un número mínimo de días, un número máximo de días, un intervalo de fechas entre las que se puede realizar ese viaje y cada uno de los viajeros que irán en el viaje (de los que se guarda nombre, edad, el estado físico -de 1 a 10-, si tienen alguna alergia o no, y si tienen movilidad reducida o no). El usuario tiene asociados todos los viajes que ha hecho en el pasado, y de cada viaje se guarda la fecha de inicio, la fecha de fin, el precio total del viaje, los viajeros que fueron en el viaje, los destinos, los medios de transporte, los alojamientos y las actividades realizadas.

Para recomendar un viaje esta empresa usa siete características:

- el **presupuesto**, que tiene en cuenta el ratio entre el precio máximo que se quiere gastar el usuario en el viaje y el número de viajeros, y que puede ser bajo (< 200 euros/persona), medio (entre 200 y

500 euros/persona) y alto (más de 500 euros/persona);

- la **tipología de viajeros**, distinguiendo entre individuo, grupo sin niños, pareja o grupo con niños;
- la **duración** del viaje, distinguiendo entre cuatro clases: de 1 a 3 días, de 4 a 7 días, de 7 a 14 días o más de 14 días;
- el **nivel de actividad** que analiza las actividades que ha realizado el usuario en sus viajes anteriores y el estado físico de los viajeros del nuevo viaje y que toma tres valores: baja (la mayoría de actividades en viajes pasados fueron calmadas o hay viajeros en la petición con movilidad reducida o menores de 6 años o mayores de 65 años), media (la mayoría de actividades en viajes pasados fueron de un nivel intermedio o hay viajeros en la petición entre los 7 y los 13 años o entre los 55 y los 65 años) o alta (en caso contrario);
- el **nivel de riesgo** que analiza el nivel de riesgo de los destinos y las actividades realizadas por el usuario en sus viajes anteriores y que toma tres valores: bajo (la media del nivel de riesgo de la mayoría de actividades y destinos anteriores está entre 1 y 3), medio (la media está entre 4 y 7) o alto (la media está entre 8 y 10);
- la **curiosidad cultural** que analiza la proporción de actividades culturales realizadas en viajes pasados respecto al total y que puede tomar tres valores: alta (más de un 70 % de las actividades anteriores fueron culturales), media (entre un 30 % y un 70 % de actividades culturales) y baja (por debajo del 30 % de actividades);
- la **temporada** del viaje, que puede ser primavera, verano, otoño o invierno, y que se obtiene a partir del rango de fechas de viaje en la petición del viajero;

A partir de estas características se determinan dos características más:

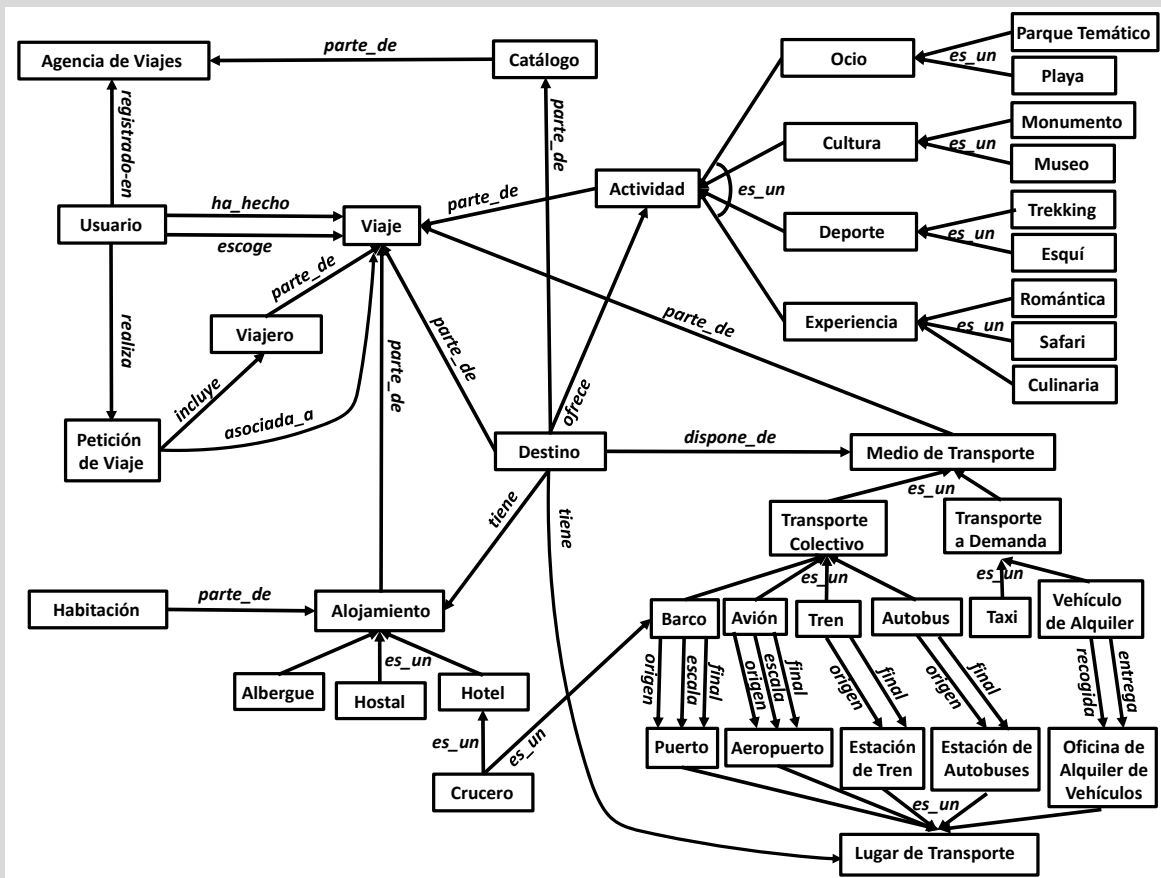
- el **arquetipo de viaje**, distinguiendo entre ocho arquetipos (entre paréntesis las características relevantes): "Viaje con niños" (para grupos con niños, actividad media o alta, riesgo bajo); "Monumentos y museos" (curiosidad cultural alta, actividad baja o media); "Sol y playa" (actividad baja o media, riesgo bajo, en cualquier temporada ya que hay destinos de playa en el mundo todo el año); "Experiencia Culinaria" (curiosidad cultural media o alta, actividad baja, riesgo bajo); "Aventura Trekking" (actividad alta, riesgo medio, temporada de primavera u otoño); "Aventura Safari" (individuo o grupo sin niños, actividad alta, riesgo alto); "Disfruta de la Nieve" (actividad alta, riesgo medio, temporada de invierno o verano, ya que será invierno en el hemisferio sur); "Viaje romántico" (una pareja de viajeros, actividad baja, riesgo bajo);
- el **destino recomendado**, distinguiendo entre 11 destinos (entre paréntesis las características relevantes): "Viaje de proximidad" (1 a 3 días o presupuesto bajo); "Viaje por Europa Central" (4 a 7 días, riesgo bajo, presupuesto medio); "Viaje por el Mediterráneo" (4 a 7 días, riesgo bajo, presupuesto medio o alto); "Viaje por el Magreb" (4 a 7 días, riesgo medio, presupuesto medio); "Viaje por Africa" (7 a 14 días o >14 días, presupuesto alto, riesgo medio o alto); "Viaje por Oriente Medio" (7 a 14 días, presupuesto alto, riesgo alto); "Viaje por Asia" (7 a 14 días o >14 días, presupuesto alto, riesgo medio, temporada no sea verano para evitar los monzones); "Viaje por Oceanía o Polinesia" (7 a 14 días o >14 días, presupuesto alto, riesgo medio); "Viaje por América del Norte" (7 a 14 días o >14 días, presupuesto alto, riesgo medio o bajo); "Viaje por América Central o del Sur" (7 a 14 días o >14 días, presupuesto alto, riesgo medio o alto); "Viaje al Caribe" (7 a 14 días, presupuesto alto, riesgo medio o bajo);

La salida del sistema es una lista de viajes recomendados que se genera llamando a la función `Viaje listar_viajes_recomendados(Arquetipos_Viaje, Destinos_recomendados, precio_maximo, min_dias; max_dias)` y se muestra por pantalla para que el usuario escoja una opción. Al obtener el viaje escogido se llama a una segunda función `Actividades listar_actividades_extra(Viaje, Arquetipos_Viaje, precio_maximo)` que muestra al usuario actividades extra que puede añadir a su viaje para finalizar su reserva.

- a) (2,5 puntos) Diseña la ontología del dominio descrito, incluyendo todos los conceptos que aparecen en la descripción e identificando los atributos más relevantes. Lista que conceptos forman parte de los datos de entrada del problema y que conceptos forman parte de la solución. Justifica las decisiones de diseño que has realizado. En el caso de las subclases de una clase, justifica con un texto breve la necesidad de crear dichas subclases. Para cada concepto de la ontología lista todos sus atributos (para cada atributo se ha de indicar su nombre y su tipo, y en el caso de las enumeraciones, sus valores posibles). [Nota: tened en cuenta que la ontología puede necesitar modificaciones para adaptarla al

apartado siguiente.]

En este problema la ontología ha de incorporar, como mínimo, todos los conceptos que forman parte de la entrada del problema (la información que se obtiene directamente de la petición del usuario y el histórico de sus viajes) y de la solución (los viajes escogidos para el usuario, cada uno con sus viajeros, su destino, su alojamiento, sus medios de transporte y sus actividades).



De la ontología resultante cabe remarcar que es importante distinguir entre **Viaje** y **Destino**, ya que en un mismo viaje se pueden visitar varios destinos, y mientras **Destino** tiene asociadas todas las actividades, alojamientos y transportes disponibles, **Viaje** asocia los que han sido escogidos. También se considera en esta solución que todos los elementos de **Viajero** (**Destino**, **Medio_de_transporte**, **Alojamiento**, **Actividad**) son componentes esenciales para definirlo, por ello se han modelado como *parte_de* un **Viaje**. También cabe remarcar que existen varios conceptos que comparten atributos y/o relaciones, y que esto ha sido algo que se ha tenido muy en cuenta a la hora de crear super-clases con las características comunes, o para decidir entre crear subclases o solo añadir un atributo **tipo**. La regla general es que hemos creado subclases cuando una o varias tienen atributos diferentes y/o relaciones diferentes. Ese es el caso de las subclases de **Alojamiento** (**Hotel** tiene una relación extra con **Crucero**), **Lugar_de_transporte** (todas las subclases se relacionan con clases diferentes, y **Aeropuerto** tiene atributos extra), **Medio_de_transporte** (todas las subclases se relacionan con clases diferentes, y muchas tienen atributos diferentes), **Actividad** (muchas subclases tienen atributos diferentes) y **Ocio** (sus dos subclases tienen atributos diferentes). Cabe destacar también que las clases **Barco**, **Avion**, **Tren** y **Autobús** están en el mismo nivel de la jerarquía de clases y comparten tantos atributos que se ha creado una superclase **Transporte_Colectivo** que define esos atributos compartidos, y para equilibrar la taxonomía se ha creado una superclase hermana **Transporte_a_demanda** de la que son subclass los taxis y los vehículos de alquiler. En el caso de las clases **Cultura** y **Parque_temático**, aunque comparten todos los atributos no se les ha creado una superclase común al no estar en el mismo nivel de la jerarquía. Las subclases de **Cultura**, **Deporte** y **Experiencia**, no tienen ni atributos ni relaciones diferentes, pero se han añadido siguiendo el heurístico del diseño de ontologías que recomienda que ramas hermanas de la taxonomía (en este

caso las actividades) tengan un nivel de granularidad similar. No hemos creado **Persona** como superclase de **Viajero** y **Usuario** porque no tienen ningún atributo común, y se puede considerar que un usuario no es una persona, sino una identidad digital que puede ser compartida por más de una persona. Pero no se ha considerado incorrecto el considerar que los usuarios son personas y por lo tanto crear una taxonomía con **Persona**, **Usuario** y **Viajero**.

Atributos: a continuación se listan los atributos mínimos para representar la información mencionada explícitamente en el enunciado y la necesaria para el apartado b) del problema. Hay otros atributos y conceptos que se deberían añadir si quisiéramos una ontología lo más general y reutilizable posible.

- **Agencia_de_Viajes:** nombre (string);
- **Usuario:** username (string), password (string encriptado), *Nivel_Actividad* (enumeración: {baja, media, alta}), *Nivel_Riesgo* (enumeración: {bajo, medio, alto}), *Curiosidad_Cultural* (enumeración: {baja, media, alta});
- **Petición:** precio_maximo (Euros), min_dias (entero positivo), max_dias (entero positivo), fecha_min (fecha), fecha_max (fecha), *Presupuesto* (enumeración: {bajo, medio, alto}), *DURACIÓN* (enumeración: {1a3, 4a7, 7a14, >14 }), *Tipología_Viajero* (enumeración: {individuo, pareja, grupo_sin_niños, grupo_con_niños}), *Temporada* (enumeración: {primavera, verano, otoño, invierno }), *ARQUETIPO_VIAJE* (enumeración: {"Viaje con niños", "Monumentos y museos", "Sol y playa", "Experiencia Culinaria", "Aventura Trekking", "Aventura Safari", "Disfruta de la Nieve", "Viaje romántico"}), *DESTINO_RECOMENDADO* (enumeración: {"Viaje de proximidad", "Viaje por Europa Central", "Viaje por el Mediterraneo", "Viaje por el Magreb", "Viaje por Africa", "Viaje por Oriente Medio", "Viaje por Asia", "Viaje por Oceania o Polinesia", "Viaje por América del Norte", "Viaje por América Central o del Sur", "Viaje al Caribe" });
- **Viajero:** nombre (string), edad (entero positivo), estado_físico (enumeración: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }), movilidad_reducida? (booleano);
- **Viaje:** fecha_inicio (fecha), fecha_fin (fecha), precio_total (euros);
- **Destino:** población (string), pais (string), moneda (string), nivel_riesgo (enumeración: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 });
- **Lugar_de_Transporte:** nombre (String), dirección (string), GPS_latitud (real), GPS_longitud (real);
- **Aeropuerto:** código_internacional (string);
- **Transporte_Colectivo:** compañía (string), hora_salida (hora), hora_llegada (hora), precio_trayecto (euros);
- **Taxi:** Dirección_origen (string), Dirección_final (string), precio_km (euros/km);
- **Vehículo_de_Alquiler:** fecha_recogida (fecha), fecha_entrega (fecha), marca (string), modelo (string), plazas (entero positivo), precio_dia (euros/dia), precio_km_extra (euros/km), precio_seguro (euros);
- **Alojamiento:** nombre (string), estrellas (enumeración: {1, 2, 3, 4, 5 });
- **Habitación:** id_habitación (string), max_personas (entero positivo), num_camas_indiv (entero positivo), num_camas_dobles (entero positivo), baño? (booleano), televisión? (booleano), wifi? (booleano), minibar? (booleano);
- **Actividad:** nombre (string), dirección (string), descripción (string), nivel_riesgo (enumeración: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 }), tipo (enumeración: {individual, pareja, grupo }), edad_min (entero positivo), edad_max (entero positivo);
- **Parque_Temático:** hora_apertura (hora), hora_cierre (hora), precio_por_persona (euros);
- **Cultura:** hora_apertura (hora), hora_cierre (hora), precio_por_persona (euros);
- **Deporte:** nivel_técnico (enumeración: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10 });
- **Experiencia:** hora_inicio (hora), duración (minutos), precio_por_persona (euros);

Como se puede ver, hemos decidido también representar las características abstractas del problema y las de la solución abstracta (son los que tienen el nombre en *Cursiva* o *CURSIVA*, respectivamente). De esta manera todos los conceptos que aparecerán en las reglas están soportados por la ontología.

- b) (2,5 puntos) El problema descrito es un problema de análisis. Explica cómo lo resolverías usando clasificación heurística. Justifica si el tipo de solución que se pide requiere de refinamiento o no. Lista las variables que forman parte del Problema Abstracto (nombre y valores posibles). Lista las variables que forman parte de la Solución Abstracta (nombre y valores posibles). Da al menos 3 ejemplos de reglas (suficientemente variadas, utilizando diferentes conceptos) para cada una de las fases de esta metodología, usando los conceptos de la ontología desarrollada en el apartado anterior. Para las reglas podéis usar la notación de alto nivel que usamos en los ejercicios de problemas (si ... entonces ...) o una notación más próxima a la del lenguaje CLIPS (siempre que queden claro los elementos del antecedente de la regla y del consecuente).

Para resolverlo mediante clasificación heurística debemos identificar en el problema las diferentes fases y elementos de esta metodología. En este caso hay solo una opción posible: la solución que pide el enunciado solo puede ser una solución concreta, ya que para poder llamar a las funciones `listar_viajes_recomendados` y `listar_actividades_extra` se necesita poder acceder a datos concretos de la petición como `precio_máximo`, `min_dias` o `max_dias`).

Una vez sabemos cuantas fases requiere nuestra solución ya podemos enunciar la solución completa. El primer elemento son los problemas concretos que hay que tratar. En este dominio los problemas concretos están definidos por toda la información detallada sobre la petición del usuario y los viajes anteriores que ha realizado.

El segundo elemento son los problemas abstractos, estos estarán definidos a partir de las 7 características que menciona el enunciado (*Presupuesto*, *Tipología_Viajeros*, *Duración*, *Nivel_Actividad*, *Nivel_Riesgo*, *Curiosidad_Cultural* y *Temporada*), todas ellas con el rango de valores discretos que aparece en el enunciado y que hemos definido en el apartado a)

Para conectar los problemas concretos con los abstractos necesitamos definir las reglas de abstracción de datos, por ejemplo:

- si (Petición.precio_maximo/cardinalidad(Petición.tiene_viajero)) < 200
 entonces Petición.Presupuesto = bajo;
- si (Petición.precio_maximo/cardinalidad(Petición.tiene_viajero)) > 500
 entonces Petición.Presupuesto = alto;
- si cardinalidad(Petición.tiene_viajero)==1 entonces Petición.Tipología_Viajeros = individuo;
- si cardinalidad(Petición.tiene_viajero.edad<18)>0 y cardinalidad(Petición.tiene_viajero)>1
 entonces Petición.Tipología_Viajeros = grupo_con_niños;
- si (Petición.max_dias) < 4 entonces Petición.Duración = "1a3";
- si (Petición.fecha_min > "15 de Junio") y (Petición.fecha_max < "20 de Septiembre")
 entonces Petición.Temporada = verano;

El tercer elemento son las soluciones abstractas. En este caso tenemos solo 2 soluciones abstractas: el arquetipo de viaje ("Viaje con niños", "Monumentos y museos", "Sol y playa", "Experiencia Culinaria", "Aventura Trekking", "Aventura Safari", "Disfruta de la Nieve" ó "Viaje romántico") y los destinos recomendados ("Viaje de proximidad", "Viaje por Europa Central", "Viaje por el Mediterraneo", "Viaje por el Magreb", "Viaje por Africa", "Viaje por Oriente Medio", "Viaje por Asia", "Viaje por Oceanía o Polinesia", "Viaje por América del Norte", "Viaje por América Central o del Sur" ó "Viaje al Caribe").

Para ligar los problemas abstractos con las soluciones abstractas necesitaremos reglas de asociación heurística, como por ejemplo:

- si Petición.Tipología_Viajeros == grupo_con_niños
 y (Usuario.Nivel_Actividad == media o Usuario.Nivel_Actividad == alta)
 y Usuario.Nivel_Riesgo == bajo
 entonces Petición.ARQUETIPO_VIAJE = "Viaje con niños";
- si (Petición.Tipología_Viajeros == individuo
 o Petición.Tipología_Viajeros == grupo_sin_niños)
 y Usuario.Nivel_Actividad == alta y Usuario.Nivel_Riesgo == alto
 entonces Petición.ARQUETIPO_VIAJE = "Aventura Safari";
- si Usuario.Nivel_Actividad == alta y Usuario.Nivel_Riesgo == medio
 y (Petición.Temporada == invierno o Petición.Temporada == verano)
 entonces Petición.ARQUETIPO_VIAJE = "Disfruta de la Nieve";
- si Petición.Duración == "1a3" o Petición.Presupuesto == bajo
 entonces Petición.DESTINO_RECOMENDADO = "Viaje de proximidad";
- si (Petición.Duración == "7a14" o Petición.Duración == ">14")

El cuarto y último elemento son las soluciones concretas. En este caso necesitamos solo una regla de producción que realice las llamadas a las dos funciones que generan la salida por pantalla para que el usuario escoja la elección final. En estas llamadas combinamos la solución abstracta con datos concretos de la petición.

```
▪ si CIERTO entonces {
    Usuario.viaje_escogido= listar_viajes_recomendados(Petición.ARQUETIPO_VIAJE,
    Petición.DESTINO_RECOMENDADO, Petición.precio_maximo, Petición.min_dias;
    Petición.max_dias);
    actividades_escogidas = listar_actividades_extra(Usuario.viaje_escogido,
    Petición.ARQUETIPO_VIAJE, Petición.precio_máximo);
    para cada Actividad A en actividades_escogidas
        añadir relacion A.parte_de(Usuario.viaje_escogido);
}
```

2. (5 puntos) La empresa de aplicaciones para móvil *SmartApps* quiere desarrollar el módulo de IA de nueva app, *ShareBath*. En los pisos pequeños con un solo cuarto de baño suele haber muchos conflictos sobre el uso del baño, especialmente por las mañanas cuando todos tienen que lavarse y arreglarse en un espacio pequeño. Por ello han pensado en una app para familias y estudiantes que comparten piso de un solo baño que les ayude a planificar el orden en el que cada persona ha de hacer sus tareas.

Cada persona se ha de duchar, peinar y usar el sanitario. Otras acciones opcionales que dependen de la persona son afeitarse, secar el pelo y maquillarse, y las tres requieren el uso del espejo. Las acciones de peinar, secar el pelo y maquillarse se han de hacer siempre después de ducharse (si hay que secar el pelo, peinar es una acción previa para desenredar el pelo y el peinado se acaba durante el secado). Cada persona tiene su propio peine, toalla, maquina de afeitar y maquillaje, pero hay otras cosas que se han de compartir como el sanitario, la ducha, el lavamanos con espejo y el secador de pelo. Para las cosas que se han de compartir nos piden que se modele explícitamente las acciones de coger y dejar ese recurso, evitando que la app recomiende cosas incompatibles como dos personas secándose el pelo con el mismo secador o una persona afeitándose y otra maquillándose delante del mismo lavamanos con espejo.

En el momento de registrarse en la app el usuario ha de hacer un inventario de los elementos del lavabo que se comparten (sanitario, ducha, lavamanos con espejo, secador) y para cada individuo ha de decir que tareas ha de realizar cada mañana (pensad que no todos los hombres necesitan afeitarse, y que el secador no lo usan solo mujeres, sino también hombres con el pelo largo).

El resultado es un plan que lista en que orden han de realizar sus tareas cada uno de los individuos dentro del baño compartido, evitando conflictos. Solo se pide un orden de las tareas, no se tiene que modelar su duración o hacer una planificación minuto a minuto.

- a) (4 puntos) Describe el dominio (incluyendo predicados, acciones, etc...) usando PDDL. Da una explicación razonada de los elementos que has escogido. Ten en cuenta que el modelo del dominio ha de poderse extender no sólo a más o menos usuarios del baño sino también a más o menos cosas compartidas (por ejemplo baños con dos lavamanos o con dos secadores de pelo).

En este problema tenemos tareas independientes que se pueden hacer en cualquier momento al no depender de una acción previa (afeitarse, ducharse, usar el sanitario) y otras que si dependen de una previa (maquillarse y peinarse tienen como antecesor bañarse, y secar el pelo tiene como antecesor peinar). Todas las tareas requieren al menos un recurso compartido, excepto secar el pelo, que requiere dos (el lavamanos con espejo y el secador). Como se indicaba en el problema, la gestión explícita de recursos compartidos es necesaria (ya que si no, el problema se convierte en algo trivial).

Existen muchas posibles formas de modelar este dominio en PDDL. Adjuntamos aquí una rúbrica de aquellas cosas que se han valorado y/o penalizado del total.

Algunas cosas que tendremos en cuenta en la solución propuesta:

- Se ha penalizado entre 0.5 y 1 punto el tener acciones individuales y distintas por cada tipo de acción en el baño: es mala praxis en PDDL, aumenta el tamaño del código e impide expandirlo para nuevos tipos de tareas, y además es totalmente innecesario. En casos normales, al no ser un problema grave, se ha penalizado en pequeña cantidad, pero en casos donde las acciones hayan sido mal modeladas, con falta de comprobaciones, o sencillamente requiriendo un número extremadamente alto de predicados, se ha penalizado hasta 1 punto.
- No modelar el obtener/dejar un ítem se ha penalizado aproximadamente en 2 puntos. Se pedía de forma explícita su modelado, y el no hacerlo trivializa el problema a una acción sencilla (solo debe mirar si tareas predecesoras ya están hechas).
- Tener acciones de obtener y/o dejar un ítem compuesto con la parte de realizar la acción. Hay diversas variantes de este caso (tener acción hacer A que requiere un recurso X libre, y luego una acción dejar que deja X libre). El problema principal de esto es que hace imposible usar un recurso dos veces seguidas (requiriendo, por ejemplo maquillarse, dejar espejo, secar el pelo, dejar espejo, peinarse...). En casos con este error, se ha penalizado hasta un punto, en casos donde no hay dicho error no se han penalizado.
- Impedir realizar acciones que requieran de más de un objeto compartido. Como se muestra en el enunciado, secarse requiere tanto de espejo como de secador. En caso de que realizar una acción compruebe solo que tengamos un objeto, esto no funciona. Se ha penalizado de 1 a 1.5 puntos.
- Modelar los objetos que no se comparten (peine, maquillaje, ...). No afectan al orden de las tareas y su introducción en el modelo solo incrementa de forma innecesaria el número de objetos con los que unificar variables y el número de predicados. No se ha penalizado, pero es incorrecto de cara al modelado.
- Otros errores no listados: no comprobar que la acción tenga precedencias, no comprobar que el ítem que un usuario tiene sea el que se pide para la tarea actual, no contabilizar quién tiene un ítem al obtenerlo, usar una constante en el fichero de dominio... Cada uno de estos se ha penalizado dependiendo del caso y del resto del modelo.

A continuación dejamos un ejemplo de solución suficiente para obtener nota máxima (a pesar de ser una solución bastante subóptima). Posteriormente listamos la explicación de predicados junto con mejoras subsiguientes para tener un modelo óptimo:

```
(define (domain sharebath)
  (:requirements :adl :typing)
  (:types persona accion itemCompartido - object
    ducha servicio lavamanos secador - itemCompartido
  )

  (:predicates
    (quiere ?p - persona ?a - accion)
    (tiene ?p - persona ?i - itemCompartido)
    (libre ?i - itemCompartido)
    (independiente ?a - accion)
    (precede ?a1 ?a2 - accion)
    (necesitaDucha ?a - accion)
    (necesitaWC ?a - accion)
    (necesitaLavamanos ?a - accion)
    (necesitaSecador ?a - accion)
  )

  (:action obtener_item
    :parameters (?p - persona ?i - itemCompartido)
    :precondition (libre ?i)
    :effect (and (not (libre ?i)) (tiene ?p ?i))
  )
)
```

```

(:action liberar_item
  :parameters (?p - persona ?i - itemCompartido)
  :precondition (tiene ?p ?i)
  :effect (and (not (tiene ?p ?i)) (libre ?i))
)

(:action realizar_accion_independiente
  :parameters (?p - persona ?a - accion)
  :precondition (and (quiere ?p ?a) (independiente ?a)
    (imply (necesitaDucha ?a) (exists (?i - ducha)(tiene ?p ?i)))
    (imply (necesitaWC ?a) (exists (?i - servicio)(tiene ?p ?i)))
    (imply (necesitaLavamanos ?a)
      (exists (?i - lavamanos)(tiene ?p ?i)))
    (imply (necesitaSecador ?a) (exists (?i - secador)(tiene ?p ?i)))
  )
  :effect (not (quiere ?p ?a))
)

(:action realizar_accion_dependiente
  :parameters (?p - persona ?a - accion ?ant - accion)
  :precondition (and (quiere ?p ?a)
    (precede ?ant ?a) (not (quiere ?p ?ant))
    (imply (necesitaDucha ?a) (exists (?i - ducha)(tiene ?p ?i)))
    (imply (necesitaWC ?a) (exists (?i - servicio)(tiene ?p ?i)))
    (imply (necesitaLavamanos ?a)
      (exists (?i - lavamanos)(tiene ?p ?i)))
    (imply (necesitaSecador ?a) (exists (?i - secador)(tiene ?p ?i)))
  )
  :effect (not (quiere ?p ?a))
)
)

```

Tenemos los siguientes predicados:

- **quiere**, que sirve para indicar que tareas quiere realizar el usuario. También nos sirve para saber si la tarea está pendiente (predicado cierto) o si ya se ha realizado (predicado falso). En este problema un usuario está servido cuando todos sus predicados **quiere** son falsos, y es así como lo expresaremos en el objetivo del problema (se ve en el apartado siguiente).
- **tiene**, que si es cierto nos dice que el usuario tiene un cierto objeto compartido en ese momento.
- **libre**, que nos dice si un objeto compartido está libre. Aunque puede parecer redundante respecto al predicado **tiene**, tener un predicado que no especifica el usuario que bloquea el objeto es muy útil por dos razones: 1) para poderlo usar como filtro en la precondición de la acción sin necesidad de añadir un **exists**; 2) para poder expresar en el objetivo del problema que al acabar todos los usuarios deben haber soltado los objetos compartidos sin necesidad de añadir un **forall** para los usuarios (se ve en el apartado siguiente).
- **precede**, que nos dice que una cierta tarea se ha de hacer antes de otra, sirve para controlar el orden de las acciones.
- **independiente**, que nos dice que esa tarea no tiene predecesor y evita la necesidad de comprobar con un **exists** la no existencia de acción predecesora. Este predicado se puede suprimir a cambio de hacer todas las tareas 'independientes' tareas que dependen de la tarea 'nada' (en el fichero de problema), eliminando también la acción. Dependiendo de la cantidad de dependencias entre tareas, esto puede ser más o menos eficiente, pero simplifica el código bastante.
- **necesitaDucha necesitaWC necesitaLavamanos necesitaSecador**: indican si una tarea necesita de cada uno de los recursos. Modelarlo de esta manera implica no poder extender un problema con nuevos items compartidos, pero se puede considerar un trade-off aceptable. Debajo adjuntamos diversas alternativas (con sus pros y cons) junto con la mejor versión posible.

Para eliminar los predicados explícitos de cada ítem compartido, podríamos modelarlo como (*necesita ?a - accion ?i - itemCompartido*). Sin embargo, al modelarlo así, es difícil entender qué ítems son necesarios para una tarea (ya que puede haber más de un ítem del mismo tipo, y más de un tipo necesario para una tarea). Para resolverlo se puede hacer de forma trivial teniendo otro tipo (*tipoRecurso*), modificando las acciones de forma que al obtener algo, se tiene tanto el recurso concreto (*ducha1*) como el *tipoRecurso* (*ducha*), y comprobar los tipos en lugar de los recursos concretos al realizar la acción. Es más, para mejorar todavía más la solución, podría sustituirse los recursos concretos por fluentes (*(cantidad_libre ?t-tipoRecurso)*), eliminando por completo el tipo anterior y simplificando sustancialmente el problema. En ambas de estas propuestas, deberíamos imbricar los *imply* de *realizar_accion* en un (*forall ?t-tipoRecurso*), ya que perdemos acceso a los tipos de recurso concretos dentro del fichero de dominio (recordad que no se pueden referenciar ni usar constantes/objects dentro del fichero de dominio al declararse dentro del fichero de problema). En ambos casos, desaparece la necesidad de utilizar el *exists* (al referenciar un tipo en lugar de un recurso concreto).

- b) (1 punto) Para probar el sistema nos dan el caso particular de una familia numerosa de 7 personas que comparte un baño con un sanitario, una ducha, dos lavamanos con espejo y un secador de pelo. En el momento del registro la familia nos ha pasado la siguiente lista de tareas para cada individuo (cuidado, la lista no está ordenada).

<i>persona</i>	<i>edad</i>	<i>tareas</i>
Juan	48	afeitarse, ducharse, peinarse, usar sanitario
Montse	45	ducharse, maquillarse, secar el pelo, peinarse, usar sanitario
Pablo	22	afeitarse, ducharse, secar el pelo, peinarse, usar sanitario
Miguel	19	afeitarse, ducharse, peinarse, usar sanitario
Jesus	17	ducharse, peinarse, usar sanitario
Martina	14	ducharse, secar el pelo, peinarse, usar sanitario
Oscar	12	ducharse, peinarse, usar sanitario

Describe este problema usando PDDL. Da una breve explicación de cómo modelas el problema.

En este caso el modelado del problema está totalmente marcado por el modelado del dominio del apartado anterior. Solo cabe destacar que, aunque el portal nos manda datos de las edades de los usuarios, nuestro módulo de IA los ignorará ya que son irrelevantes a la hora de construir el plan.

```
(define (problem 7pers1ducha1sanitario2lavamanos1secador)
  (:domain sharebath)
  (:objects Juan Montse Pablo Miguel Jesus Martina Oscar - persona
    afeitarse ducharse peinarse secarPelo maquillarse usarWC - accion
    ducha1 - ducha
    WC1 - servicio
    lavamanos1 lavamanos2 - lavamanos
    secador1 - secador
  )

  (:init
    (independiente ducharse) (independiente usarWC) (independiente afeitarse)
    (precede ducharse maquillarse) (precede ducharse peinarse) (precede peinarse secarPelo)
    (necesitaDucha ducharse) (necesitaWC usarWC)
    (necesitaLavamanos afeitarse) (necesitaLavamanos maquillarse)
    (necesitaLavamanos peinarse) (necesitaLavamanos secarPelo)
    (necesitaSecador secarPelo)
    (quiere Juan afeitarse) (quiere Juan ducharse)
```

```

    (quiere Juan peinarse) (quiere Juan usarWC)
    (quiere Montse ducharse) (quiere Montse peinarse) (quiere Montse maquillarse)
    (quiere Montse secarPelo) (quiere Montse usarWC)
    (quiere Pablo ducharse) (quiere Pablo peinarse) (quiere Pablo secarPelo)
    (quiere Pablo afeitarse) (quiere Pablo usarWC)
    (quiere Miguel ducharse) (quiere Miguel peinarse)
    (quiere Miguel afeitarse) (quiere Miguel usarWC)
    (quiere Jesus ducharse) (quiere Jesus peinarse) (quiere Jesus usarWC)
    (quiere Martina ducharse) (quiere Martina peinarse)
    (quiere Martina secarPelo) (quiere Martina usarWC)
    (quiere Oscar ducharse) (quiere Oscar peinarse) (quiere Oscar usarWC)
    (libre ducha1) (libre WC1) (libre lavamanos1) (libre lavamanos2) (libre secador1)
)

(:goal (and (forall (?p - persona) (not (exists (?a - accion) (quiere ?p ?a) )))
            (forall (?i - itemCompartido) (libre ?i))
        )
)
)
)

```

Se ha creado un objeto por cada persona, objeto compartido y acción, aprovechando los tipos definidos en el dominio.

El estado inicial esta compuesto por cuatro bloques de predicados:

- el primero, en el que listamos las acciones que no dependen de otra anterior
- el segundo, en el que modelamos de forma discreta las precedencias entre las acciones dentro del baño
- el tercero, en el que modelamos el tipo de objetos compartidos que necesita cada acción
- el cuarto, en el que modelamos que acciones necesita hacer cada uno de los usuarios
- el último, en el que indicamos que todos los elementos compartidos estan libres.

El estado objetivo consta de dos partes. La primera parte expresa el objetivo principal (todos los usuarios han realizado todas las acciones) buscando la no-existencia de un predicado **quiere** con valor positivo. La segunda parte fuerza a que en el estado objetivo se hayan liberado todos los objetos compartidos para evitar así que haya usuarios que han hecho todas las acciones pero no han liberado el recurso.

Las notas se publicaran el día **22 de enero**.